

**TITLE :** Write a Java program to demonstrate the implementation of interfaces.

## Problem Statement

Write a Java program to demonstrate the implementation of interfaces.

## Learning Objectives

To implement interfaces for achieving abstraction and multiple inheritance of type.

## Theory

An interface specifies a set of abstract methods that implementing classes must define. It provides complete abstraction and supports multiple inheritance of type. Interfaces improve modularity, loose coupling, and software flexibility. They allow different classes to implement common behavior while maintaining independent implementations. Interfaces are widely used in enterprise application development.

## Conclusion

This Java program successfully demonstrates the practical implementation of interfaces to achieve 100% abstraction and modular design. By defining contracts such as Printable and Switchable, completely unrelated classes—such as Student, Employee, Light, and Fan—can implement common behaviors through their own independent logic. Leveraging interfaces promotes loose coupling, facilitates dynamic method dispatch (polymorphism), and provides a clean structural framework for building scalable and maintainable applications.

## Exercise

1. Create an interface **Printable** and implement it in Student and Employee classes.

Code:

```
1 interface Printable {
2     void printDetails();
3 }
4 class Student implements Printable {
5     private String name;
6     private int rollNo;
7     public Student(String name, int rollNo) {
8         this.name = name;
9         this.rollNo = rollNo;
10    }
11    @Override
12    public void printDetails() {
13        System.out.println("Student Details -> Name: " + name + " | Roll No: " + rollNo);
14    }
15 }
16 class Employee implements Printable {
17     private String name;
18     private int empId;
19     public Employee(String name, int empId) {
20         this.name = name;
21         this.empId = empId;
22    }
23    @Override
24    public void printDetails() {
25        System.out.println("Employee Details -> Name: " + name + " | Employee ID: " + empId);
26    }
27 }
28 public class MainPrintable {
29     Run main | Debug main
30     public static void main(String[] args) {
31         Printable student = new Student("Alice", 101);
32         Printable employee = new Employee("Bob", 5001);
33         student.printDetails();
34         employee.printDetails();
35     }
36 }
```

Output:

```
Listening on 60068
User program running
Student Details -> Name: Alice | Roll No: 101
Employee Details -> Name: Bob | Employee ID: 5001
User program finished
```

2. Create an interface **Switchable** with a method `turnOn()`. Implement the interface in **Light** and **Fan** classes to display the device status.

Code:

```
1  interface Switchable {
2      void turnOn();
3  }
4  class Light implements Switchable {
5      @Override
6      public void turnOn() {
7          System.out.println("Light is now turned ON. Brightness set to 100%.");
8      }
9  }
10 class Fan implements Switchable {
11     @Override
12     public void turnOn() {
13         System.out.println("Fan is now turned ON. Spinning at medium speed.");
14     }
15 }
16 public class MainSwitchable {
17     Run main | Debug main
18     public static void main(String[] args) {
19         Switchable light = new Light();
20         Switchable fan = new Fan();
21
22         light.turnOn();
23         fan.turnOn();
24     }
25 }
```

Output:

```
Listening on 59176
User program running
User program finished
Light is now turned ON. Brightness set to 100%.
Fan is now turned ON. Spinning at medium speed.
```